T04 Melee Attack

CRE132 worksheet

What you'll make

A sword swing for the top-down player. Face a slime and press the attack button: the player swings in the direction it faces, and the slime turns redder with each hit. Three hits and it's gone. You can't swing faster than twice a second, and you stand still while you swing.

By the end you'll know how **Animation Events** let a clip call your code on exactly the right frame, how to switch a hitbox on and off, and how a list stops one swing hitting the same thing twice.

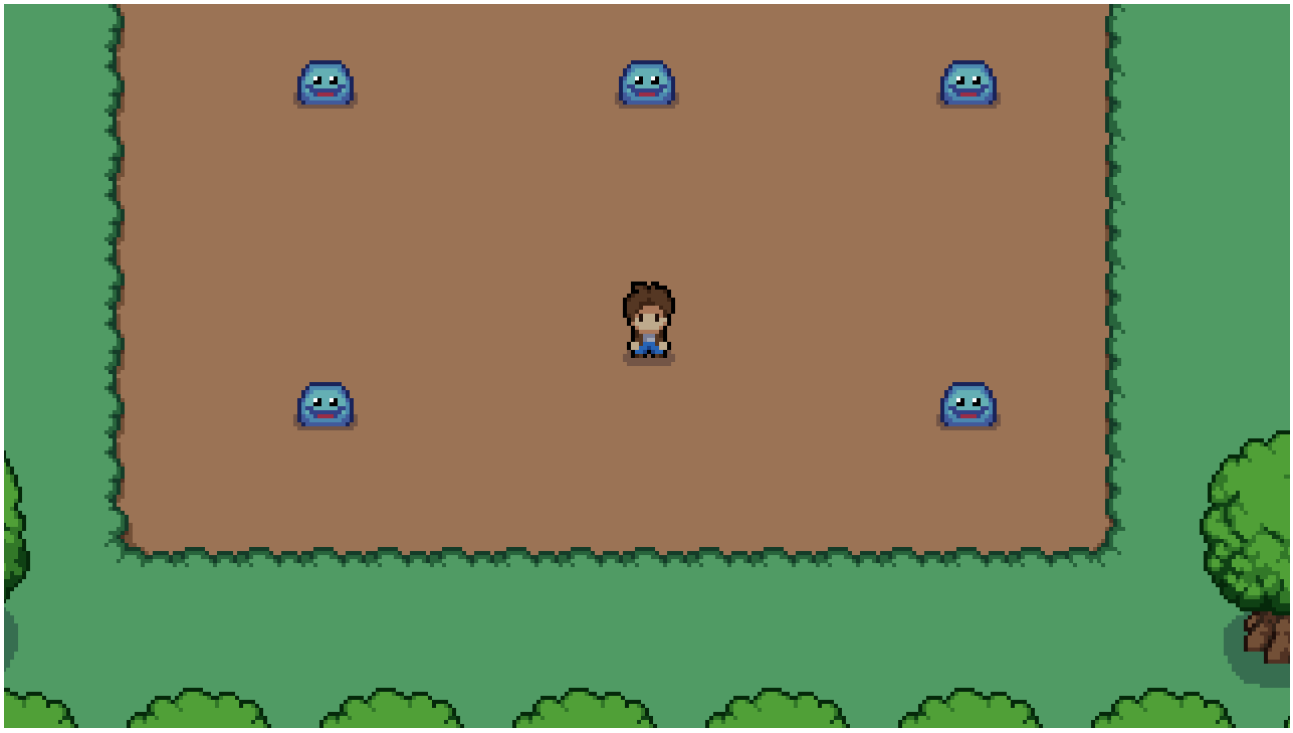


Figure 1: The finished clearing: the player in the middle of five slime dummies.

Before you start

1. Unzip `CRE132_T04_MeleeAttack_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T04_MeleeAttack_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T04_MeleeAttack`.
3. Press **Play**. You can walk in eight directions with the right animations (T01 and T02), but the attack button does nothing yet. Press Play again to stop.

Under **Characters** are the **Player** and five **Slime Dummies**, each with S01's **Health** (3) and **HealthTint**. The Player is built like T03's: the root at the feet, with a **Sprite** child holding the Animator. The Sprite has a child of its own, **Sword Hitbox**, with a **Circle Collider 2D**.

The Animator uses the `PlayerMelee` controller, which already has an **Attack** state and four attack clips. The clips don't have any Animation Events yet: you'll add them in part 4, once the methods they call exist. You'll also write the hitbox's script and the attack.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The Attack state

Select the Player's **Sprite**, open **Window > Animation > Animation** and pick **Player_Attack_Down** from the clip list at the top left. There are four attack clips (**Down**, **Right**, **Left** and **Up**), each 4 frames at 12 frames a second, so a swing lasts a third of a second. They don't loop: a swing plays once.

Now open the **Animator** window (**Window > Animation > Animator**) with **Sprite** still selected. The `PlayerMelee` controller has T02's **Idle** and **Walk**, plus an **Attack** state: a 2D blend tree of the four **Player_Attack_** clips, on **DirectionX** and **DirectionY** like the others. Click each arrow to see its settings:

- **Any State to Attack** when the **Attack** trigger is set. A **trigger** is a parameter that switches itself off once a transition has used it, which suits a one-off action like a swing. From **Any State** means you can attack whether you're standing or walking.
- **Attack to Idle** uses **Has Exit Time**, so it goes back when the clip has finished.

2. The hitbox: AttackHitbox

Make a folder `Assets/_game/Scripts/T04_MeleeAttack` and create `AttackHitbox` in it:

```
// The sword's hit area. While it's switched on, anything with Health that it touches
// takes damage, but only once per swing, even if it stays in the area for several frames.
using System.Collections.Generic;
using UnityEngine;

public class AttackHitbox : MonoBehaviour
{
    [SerializeField] private int damage = 1;

    private List<Health> alreadyHit = new List<Health>();

    // Switched on at the start of each swing: forget who was hit last time.
    private void OnEnable()
    {
        alreadyHit.Clear();
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        // InParent finds the Health on the object or above it (a character's Health is on
        // its root).
        Health health = other.GetComponentInParent<Health>();
        if (health == null || alreadyHit.Contains(health))
        {
            return;
        }

        // Don't hit ourselves.
        if (health.transform == transform.root)
        {
            return;
        }

        alreadyHit.Add(health);
        health.TakeDamage(damage);
```

```
    }
}
```

- **`OnEnable` runs every time the object is switched on**, which is the start of every swing. Clearing the list there means each swing starts fresh.
- **`alreadyHit` remembers who this swing has hurt.** A trigger can stay touching a slime for several frames. Without the list, one swing could hit it more than once.
- **`GetComponentInParent<Health>()`** looks on the collider's object and then up through its parents. Health goes on a character's root, and the collider might be on a child.
- **`transform.root`** is the top object of the hitbox's hierarchy, so the sword never hurts the one swinging it.

Add it: select **Sword Hitbox** (under **Player > Sprite**) and add **Attack Hitbox**. Leave **Damage** at **1**. It already has a **Circle Collider 2D** with **Radius** 0.6 and **Is Trigger** ticked.

3. The swing: MeleeAttack

Create `MeleeAttack` in the same `T04_MeleeAttack` folder:

```
// Press Attack to swing a sword the way the character faces. The swing's animation clips
// call HitboxOn and HitboxOff with Animation Events, so the hit happens on the frames
// where the sword is out. Put it on the object with the Animator (the events call methods
// on that object).
using System.Collections;
using UnityEngine;
using UnityEngine.InputSystem;

public class MeleeAttack : MonoBehaviour
{
    [SerializeField] private InputActionReference attackAction;
    [SerializeField] private float cooldown = 0.5f;

    // How long the character stands still for a swing (the length of the attack clips).
    [SerializeField] private float attackTime = 0.35f;

    // The sword's hit area: a child object with a trigger collider and AttackHitbox,
    // switched off between swings.
    [SerializeField] private GameObject hitbox;
    [SerializeField] private float hitboxDistance = 0.8f;
    // Where swings start from, measured from this object: low on the body, because in a
    // top-down game characters' colliders are at their feet.
    [SerializeField] private Vector2 swingOrigin = new Vector2(0f, -0.8f);

    private Animator animator;
    private DirectionalAnimator directionalAnimator;
    private TopDownMovement movement;
    private float nextAttackTime;

    private void Awake()
    {
        animator = GetComponent<Animator>();
        directionalAnimator = GetComponent<DirectionalAnimator>();
        movement = GetComponentInParent<TopDownMovement>();
        hitbox.SetActive(false);
    }

    private void OnEnable()
    {
```

```
        attackAction.action.Enable();
    }

    private void OnDisable()
    {
        attackAction.action.Disable();
    }

    private void Update()
    {
        if (attackAction.action.WasPressedThisFrame() && Time.time >= nextAttackTime)
        {
            StartCoroutine(Attack());
        }
    }

    private IEnumerator Attack()
    {
        // The cooldown stops button-mashing from swinging faster than the animation.
        nextAttackTime = Time.time + cooldown;
        movement.enabled = false;
        animator.SetTrigger("Attack");

        yield return new WaitForSeconds(attackTime);

        movement.enabled = true;
    }

    // Called by an Animation Event on the swing frame.
    public void HitboxOn()
    {
        Vector2 offset = swingOrigin + directionalAnimator.LastDirection * hitboxDistance;
        hitbox.transform.localPosition = offset;
        hitbox.SetActive(true);
    }

    // Called by an Animation Event when the swing is over.
    public void HitboxOff()
    {
        hitbox.SetActive(false);
    }
}
```

What's going on:

- **`Update` checks two things**: was Attack pressed, and has the cooldown passed? `nextAttackTime` works like the fire's timer in S01.
- **`Attack` is a coroutine** (`IEnumerator` with `yield return`). It can wait part-way through: switch movement off, start the animation, wait `attackTime` seconds, then switch movement back on. T01's `TopDownMovement` stops the body in its `OnDisable`, so you stand still while swinging.
- **`animator.SetTrigger("Attack")`** sets the trigger from part 1. The name must match exactly.
- **`HitboxOn` and `HitboxOff` are public** so the Animation Events can call them. `HitboxOn` moves the hitbox in front of the player first: the swing origin plus T02's `LastDirection` times the distance. Facing left, `LastDirection` is (-1, 0), so the hitbox goes 0.8 units to the left.
- **The swing origin is low**, (0, −0.8) from **Sprite**, near the feet. In a top-down game colliders are at the feet, so a swing at chest height would pass over the slimes' colliders.

Add it: select **Sprite** (under the **Player**) and add **Melee Attack**. Fill in:

- **Attack Action**: **Player/Attack** (J, the left mouse button or the gamepad's East button).
- **Hitbox**: drag **Sword Hitbox** from the Hierarchy into the slot.
- Leave **Cooldown** 0.5, **Attack Time** 0.35, **Hitbox Distance** 0.8 and **Swing Origin** (0, −0.8).

Test it: press Play, walk up to a slime, face it and press **J**. The player swings the right way, but the slime doesn't get hurt. `HitboxOn` is never called, so the hitbox stays switched off: the clips need Animation Events. Try mashing J: you still swing only twice a second.

4. Animation Events

An **Animation Event** calls a method when a clip reaches a certain frame. Each attack clip needs two: **HitboxOn** on frame 1, when the sword comes out, and **HitboxOff** on frame 3, when the swing is over.

1. Select **Sprite** (under the **Player**). In the Animation window, pick **Player_Attack_Down** from the clip list.
2. Type **1** in the frame number box at the top left of the Animation window, next to the play buttons. The white playhead line moves to frame 1.
3. Click the **Add Event** button. It's to the right of the frame number box: a small marker shape with a **+**. A white marker appears on frame 1, in the thin strip just above the tracks.
4. With the marker selected, the Inspector shows the event. Open the **Function** drop-down and choose **HitboxOn**. It lists the public methods of the scripts on **Sprite**, which is why you added **Melee Attack** first.
5. Type **3** in the frame number box, click **Add Event** again and choose **HitboxOff**.
6. Do steps 2 to 5 for **Player_Attack_Right**, **Player_Attack_Left** and **Player_Attack_Up**.

Click a marker to check it: the Inspector shows which method it calls. To move one, drag it along the strip; to remove one, right-click it and choose **Delete**.

- **The hit happens when the sword is drawn out**, not when the button is pressed. Change the clip and the timing follows.
- **Events call methods on scripts on the same object as the Animator.** That's why the attack script is on **Sprite**, not on the Player root.

Test it: press Play, face a slime and press **J**. The slime gets redder; the third hit removes it. Swing at a slime from each side to check all four clips.

5. Watching the hitbox

Press Play, arrange the Scene view next to the Game view and select **Sword Hitbox** in the Hierarchy. Swing in each direction: the green circle of its collider appears in front of the player for a few frames, then disappears. Between swings it's switched off (greyed out in the Hierarchy).

Check your work

- ☐ Each of the four attack clips has **HitboxOn** on frame 1 and **HitboxOff** on frame 3.
- ☐ J swings in the direction the player faces, with the right animation.
- ☐ Each hit turns a slime redder; the third removes it.
- ☐ One swing never hits the same slime twice.
- ☐ The player stands still while swinging and can't swing faster than every half second.
- ☐ Compare with `CRE132_T04_MeleeAttack_Finished` if something behaves differently.

Key points

- **Animation Events keep the hit in sync with the picture.** Change the clip and the timing follows.
- **A cooldown and standing still while attacking** make the attack feel deliberate.

- **Hitting once per swing needs memory** (the list), because a trigger can stay touching for several frames.

If something goes wrong

- **“AnimationEvent ‘HitboxOn’ has no receiver”: MeleeAttack** is on the Player root instead of **Sprite**. It must be on the object with the Animator.
- **“AnimationEvent has no function name specified”:** an event’s **Function** was left empty. Click each marker and pick the method.
- **The Function drop-down doesn’t list HitboxOn: Sprite** isn’t selected, or **Melee Attack** isn’t on it yet, or the script has a compile error. Fix the first red error in the Console first.
- **NullReferenceException in MeleeAttack’s Awake:** the **Hitbox** slot is empty.
- **NullReferenceException in Update:** the **Attack Action** slot is empty.
- **The player swings but nothing gets hurt:** that clip is missing its **HitboxOn** event. Otherwise, check **Sword Hitbox** has **Attack Hitbox**, and its **Circle Collider 2D** has **Is Trigger** ticked.
- **The green circle stays on after a swing:** that clip is missing its **HitboxOff** event.
- **The swing never plays:** check the spelling in `SetTrigger("Attack")` matches the **Attack** parameter in the Animator.

Try it

1. Move the **HitboxOn** event to frame 0 or 2 in one clip. Does the hit feel early or late compared with the picture?
2. Knock the slime back a little when it’s hit. (T08 does this properly.)
3. Make a stronger second swing if Attack is pressed again during the first: a simple combo.
4. Play S05’s slash sound from `HitboxOn`.